CitiusTech

Powering the Future of Healthcare

Docker Fundamentals

Version 1.0
November 2023

This document is confidential and contains proprietary information, including trade secrets of CitiusTech. Neither the document nor any of the information contained in it may be reproduced or disclosed to any unauthorized person under any circumstances without the express written permission of CitiusTech.

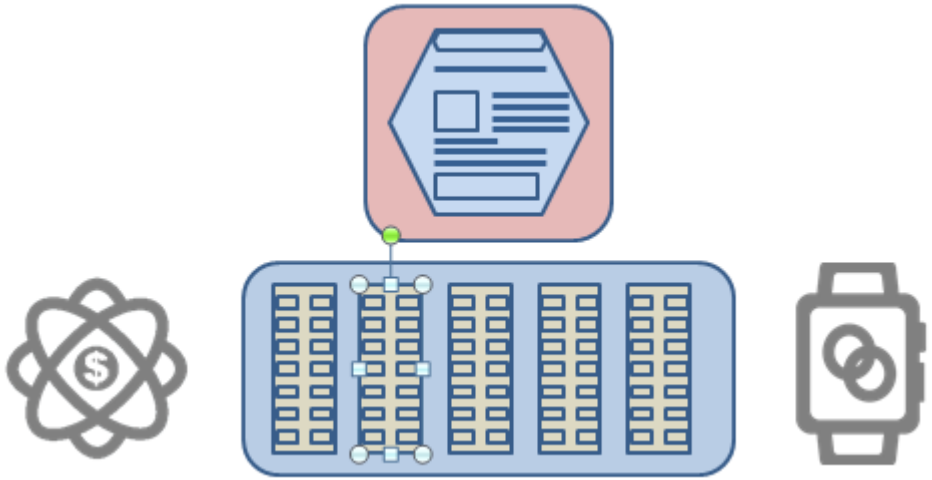
© 2021 CitiusTech | Private & Confidential

Agenda

- **Hypervisors**
- Containers and its Benefits
- Docker
- Role of Images and Container
- Docker Instruction, Hub and Repositories
- Docker Networking
- Binding and Linking Containers
- Docker Compose and Managing Data in Docker
- Docker Swarm and Service

History

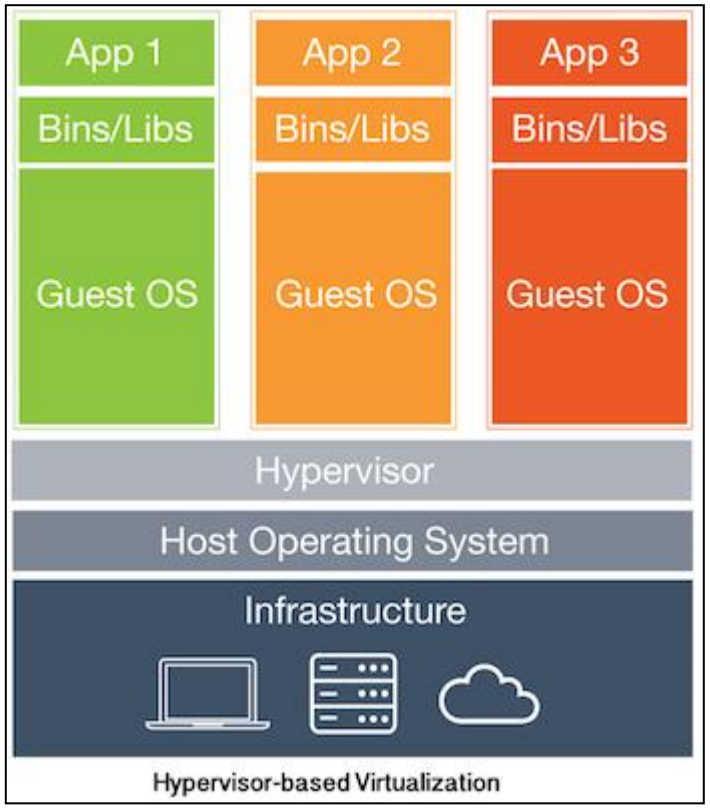
- Applications were built on Single servers. Hardware was underutilized
- We ended up with a massively overpowered physical servers running at like 10-20% of what they are capable of wasting a lot of company's capital and resources
- Solution to utilize machines capacity **– Use of Hypervisors**



- **Applications run businesses**
- **Applications run on servers**
  - Procurement lead times
  - Up-front capex
  - Ongoing opex

Importance of Hypervisors (1/2)

- Hypervisor is responsible to create a virtual hardware and software environment to run and manages Virtual Machines.
- A computer on which a hypervisor runs one or more virtual machines is called a host machine, and each virtual machine is called a guest machine.



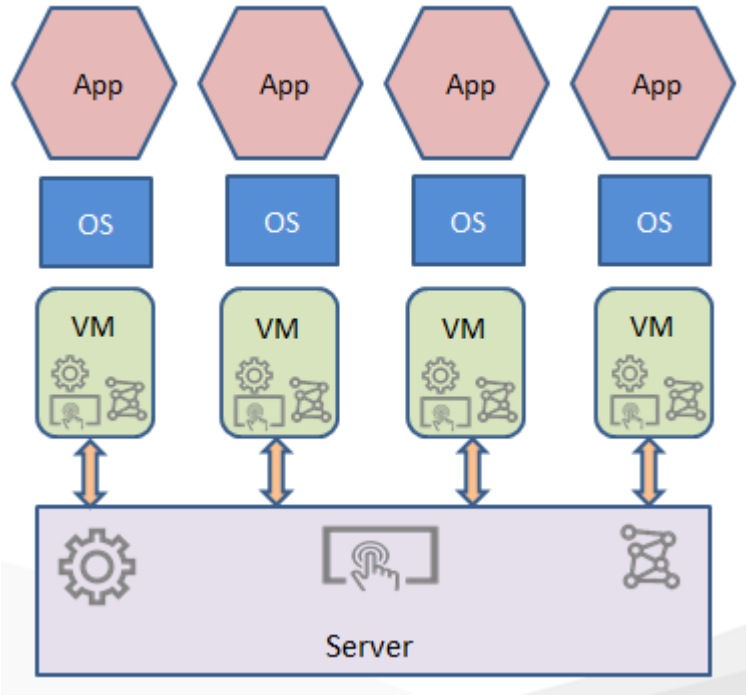
- **Hypervisors allow multiple apps per server**
- **Each OS**
  - Uses CPU
  - Uses RAM
  - Uses disk
  - May have license cost

https://blog.mikesir87.io/2017/05/docker-is-not-a-hypervisor/

Importance of Hypervisors (2/2)

Problem:

- You have entire kernel space & Application space of the Host Operating system, and we are not utilizing it.
- On top of that we are putting another Kernel Space and Another Application space.



- **Applications run businesses**
- **Applications run on servers**
  - Procurement lead times
  - Up-front capex
  - Ongoing opex
- **Hypervisors allow multiple apps per server**

Limitation of Hypervisors

Heavy

- Virtual Machines usually execute in a heavy isolated process, because it needs an entire Guest OS.
- Each VM has complete OS that consumes CPU, RAM and disk space.

High cost of implementation

- Hardware and software are required at some point and that means devices must either be developed, manufactured, or purchased for implementation.

High Maintenance

- It required backup, patching, and monitoring and in few cases licensing of OS to run on VM.

Portability

- Moving virtual machines among different types of hypervisor’s or migrating them to cloud platforms was not easy.

Agenda

- Hypervisors
- **Containers and its Benefits**
- Docker
- Role of Images and Container
- Docker Instruction, Hub and Repositories
- Docker Networking
- Binding and Linking Containers
- Docker Compose and Managing Data in Docker
- Docker Swarm and Service

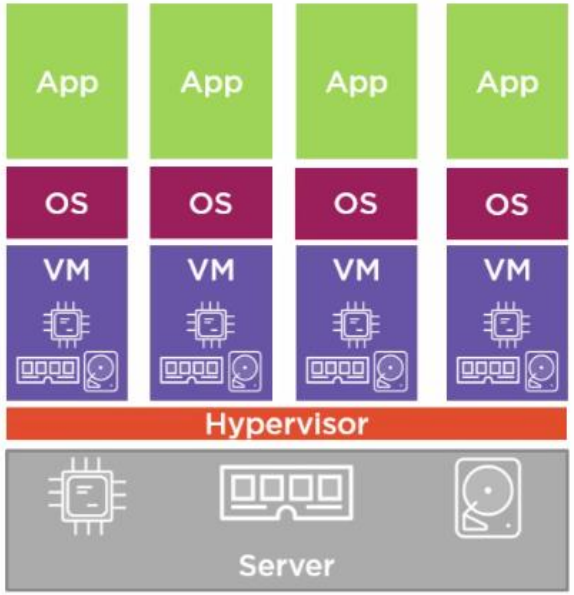
What are Containers?

Hypervisor Architecture

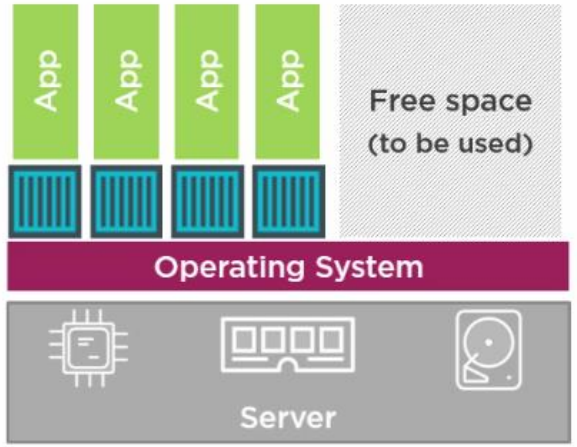
- Hungry operating system model (Eating admin time system resources, budgets)
- Every OS is prone to chunky attack Service too

Container Architecture (Just one Operating system, faster App)

- More Free space means more applications for the business too
- Perfect for situations where we need to spin things very quickly



Hypervisor Architecture



Container Architecture

Benefits of Containers

Flexible:

- Even the most complex applications can be containerized

Lightweight:

- Containers leverage and share the host kernel

Interchangeable:

- You can deploy updates and upgrades on-the-fly

Portable:

- You can build locally, deploy to the cloud, and run anywhere

Scalable:

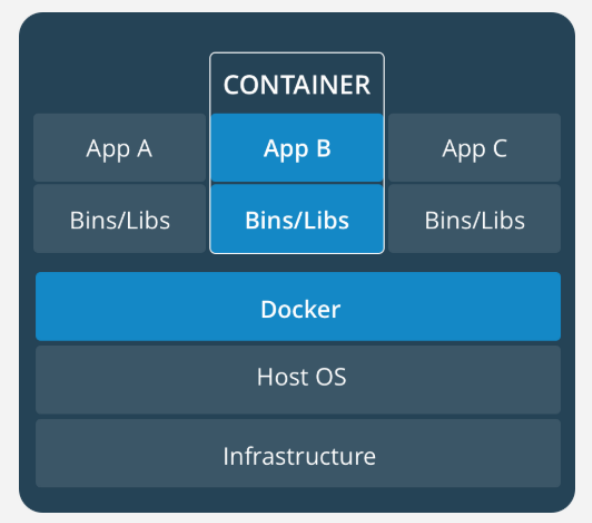
- You can increase and automatically distribute container replicas

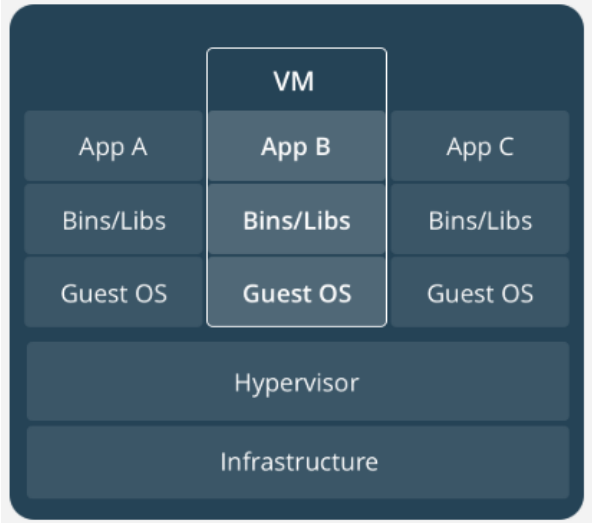
Stackable:

- You can stack services vertically and on-the-fly

Comparing Containers and Virtual Machines

Containers	Virtual machines
Containers are an abstraction at the app layer that packages code and dependencies together	Virtual machines (VMs) are an abstraction of physical hardware turning one server into many servers
Multiple containers can run on the same machine and share the OS kernel with other containers, each running as isolated processes in user space	The hypervisor allows multiple VMs to run on a single machine. Each VM includes a full copy of an operating system, one or more apps, necessary binaries and libraries
Containers start instantly	VMs are slow to boot





Agenda

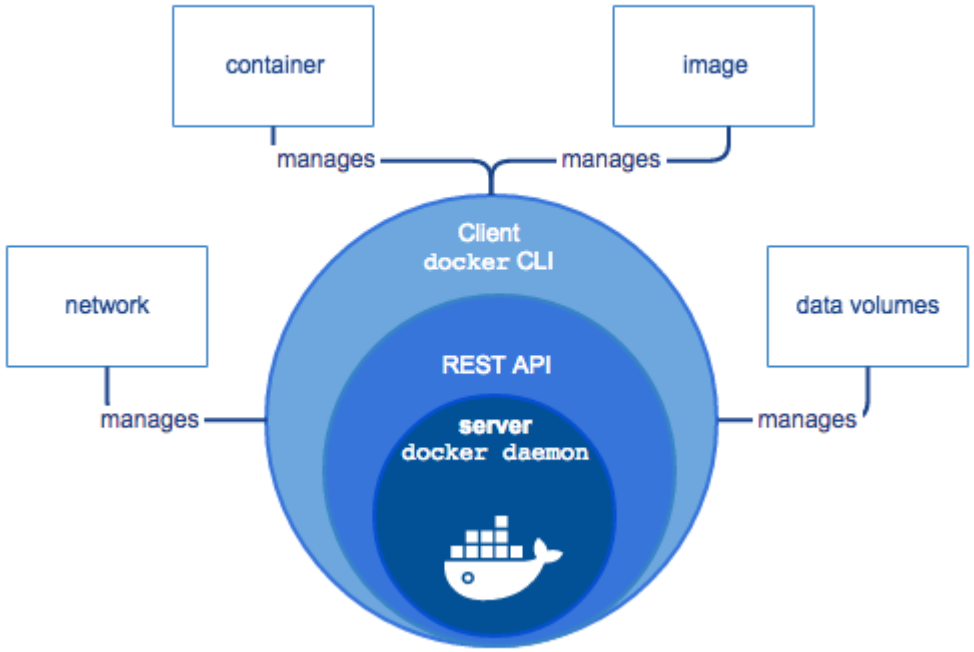
- Hypervisors
- Containers and its Benefits
- **Docker**
- Role of Images and Container
- Docker Instruction, Hub and Repositories
- Docker Networking
- Binding and Linking Containers
- Docker Compose and Managing Data in Docker
- Docker Swarm and Service

What is Docker?

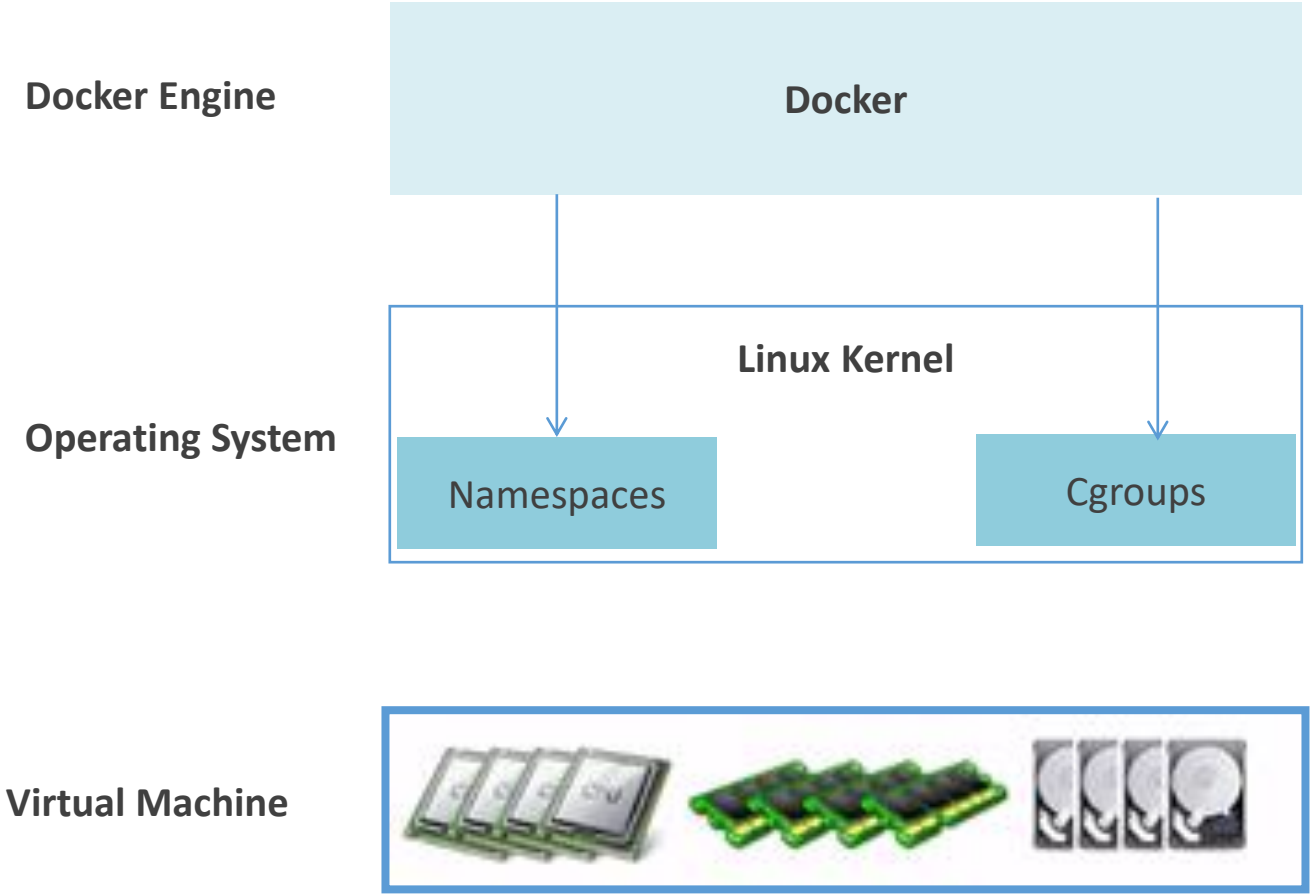
- “Docker is an open-source platform for developers and system admins to build, ship, and run containerized applications.“
- Features :
  - Container technology written in Go Language and has a huge community adoption
  - Has its own tool to run multiple containers and manage a cluster of containers easily
  - Originally released in March 2013 by dotCloud Inc. (became Docker, Inc in October 2013)
  - Announced Docker 1.0 in June 2014
  - Alliances with Red Hat(RHEL Atomic and OpenShift) and other OS developers , Google, Amazon, and other cloud providers

Docker Engine (1/2)

- Docker Engine is a client-server application with these major components:
  - A server which is a type of long-running program called a daemon process (the dockerd comand). The daemon creates and manages Docker objects, such as images, containers, network and volumes.
  - A REST API which specifies interfaces that programs can use to talk to the daemon and instruct it what to do.
  - A command line interface (CLI) client (the docker command).



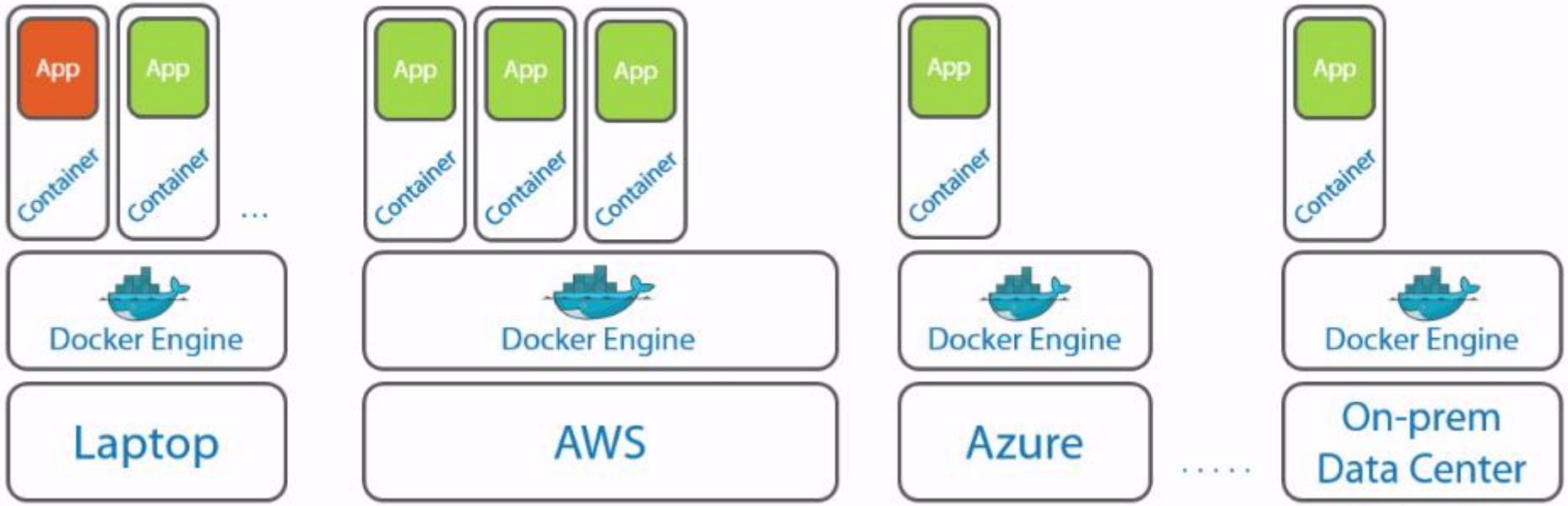
Docker Engine (2/2)



- Analogy - Docker Engine is to containers what Kernal is to RedHat/Linux

Portable Deployment with Docker

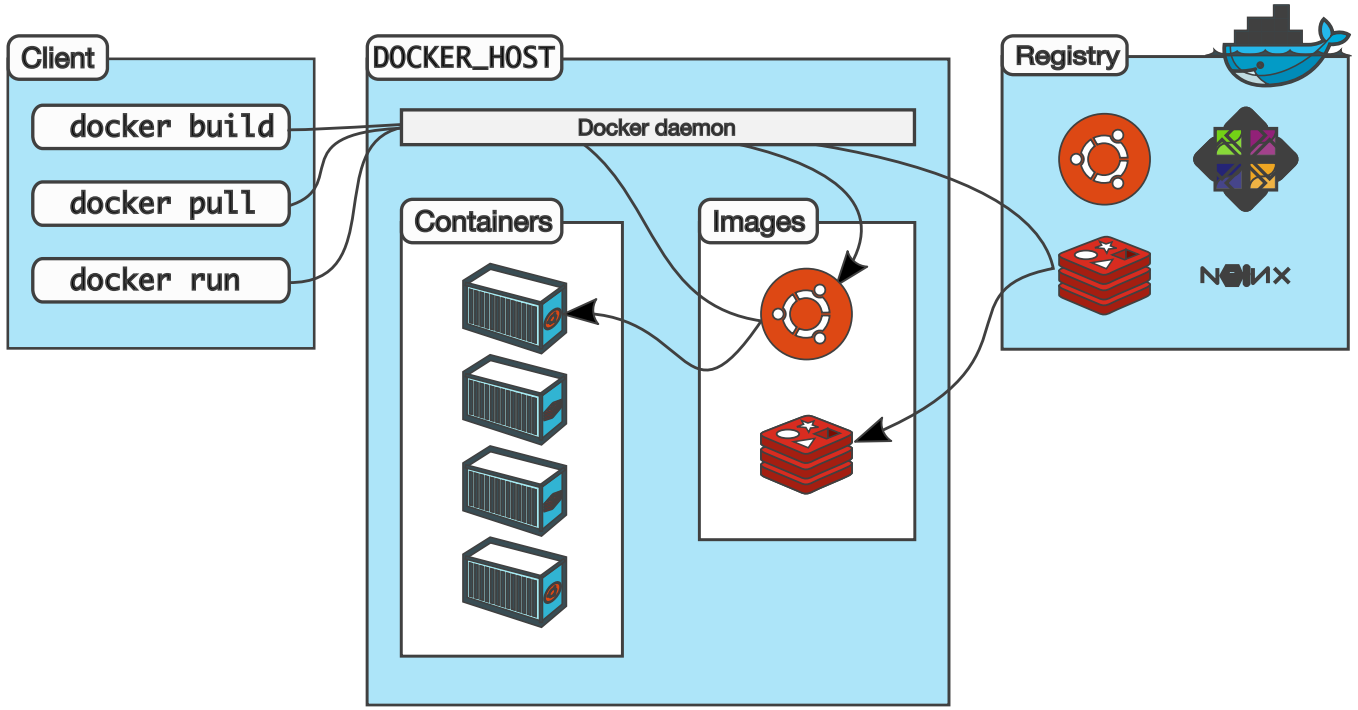
Docker defines an abstraction for these machine-specific settings. The exact same Docker container can run - unchanged - on many different machines, with many different configurations.



http://contiv.ciscolive.com/pod2/print

Docker Architecture

- Docker uses a client-server architecture. The Docker client talks to the Docker daemon, which does the heavy lifting of building, running, and distributing your Docker containers.
- The Docker client and daemon can run on the same system, or you can connect a Docker client to a remote Docker daemon. The Docker client and daemon communicate using a REST API, over UNIX sockets or a network interface.
- **Registry** : Thinks of Registry as SAAS where anybody and everybody is building an Image and pushing it over there.



https://docs.docker.com/engine/docker-overview/

Docker Architecture: Major Components (1/2)

The Docker Daemon:

- The Docker daemon (dockerd) listens for Docker API requests and manages Docker objects such as images, containers, networks and volumes.
- A daemon can also communicate with other daemons to manage Docker services.

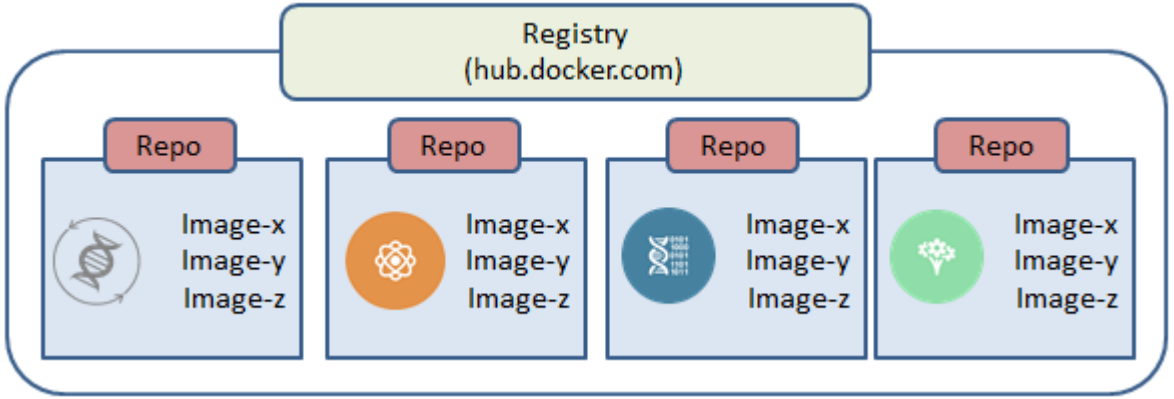
The Docker Client:

- The Docker client (docker) is the primary way that many Docker users interact with Docker. When you use commands such as docker run, the client sends these commands to dockerd, which carries them out.
- The docker command uses the Docker API. The Docker client can communicate with more than one daemon.

Docker Architecture: Major Components (2/2)

Docker Registries:

- A Docker registry stores Docker images. Docker Hub and Docker Cloud are public registries that anyone can use, and Docker is configured to look for images on Docker Hub by default.
- You can even run your own private registry. If you use Docker Datacenter (DDC), it includes Docker Trusted Registry (DTR).
- When you use the docker pull or docker run commands, the required images are pulled from your configured registry. When you use the docker push command, your image is pushed to your configured registry.



Basic Docker Commands

Few basic commands that can be executed from CLI

Command	Description
docker version	The command simply returns the version of your installed docker client and server
docker info	The command displays system-wide information
docker ps –a	List information about stopped, exited as well as running containers
docker system df or docker system df –v	Show docker disk usage

Docker Objects

- When you use Docker, you are creating and using images, containers, networks, volumes, plugins, and other objects.
- Following table describes these objects:

Object	Description
Images	An image is a read-only template with instructions for creating a Docker container
Containers	A container is a runnable instance of an image. You can create, start, stop, move, or delete a container using the Docker API or CLI
Services	Services allow you to scale containers across multiple Docker daemons, which all work together as a swarm with multiple managers and workers
Networks	Docker includes support for networking containers through the use of network drivers
Volumes	Volumes are the preferred mechanism for persisting data generated by and used by Docker containers

Docker Underlying Technology (1/2)

- Docker uses several features of Linux kernel to deliver its functionality
- Namespaces
  - Docker uses a technology called namespaces to provide the isolated workspace called the container
  - When you run a container, Docker creates a set of namespaces for that container
  - These namespaces provide a layer of isolation
  - Each aspect of a container runs in a separate namespace and its access is limited to that namespace
  - Docker Engine uses namespaces such as the following on Linux:
    - The pid namespace: Process isolation (PID: Process ID)
    - The net namespace: Managing network interfaces (NET: Networking)
    - The ipc namespace: Managing access to IPC resources (IPC: InterProcess Communication)
    - The mnt namespace: Managing filesystem mount points (MNT: Mount)
    - The uts namespace: Isolating kernel and version identifiers. (UTS: Unix Timesharing System)

Docker Underlying Technology (2/2)

- **Control Groups**
  - Docker Engine on Linux also relies on another technology called control groups (cgroups)
  - A cgroup limits an application to a specific set of resources
  - Control groups allow Docker Engine to share available hardware resources to containers and optionally enforce limits and constraints
  - For example, you can limit the memory available to a specific container
- **Union File Systems**
  - Union file systems, or UnionFS, are file systems that operate by creating layers, making them very lightweight and fast
  - Docker Engine uses UnionFS to provide the building blocks for containers
  - Docker Engine can use multiple UnionFS variants, including AUFS, btrfs, vfs, and DeviceMapper

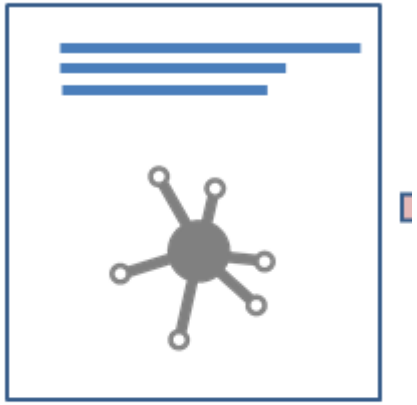
Docker Engine combines the namespaces, control groups, and UnionFS into a wrapper called container format

Agenda

- Hypervisors
- Containers and its Benefits
- Docker
- **Role of Images and Container**
- Docker Instruction, Hub and Repositories
- Docker Networking
- Binding and Linking Containers
- Docker Compose and Managing Data in Docker
- Docker Swarm and Service

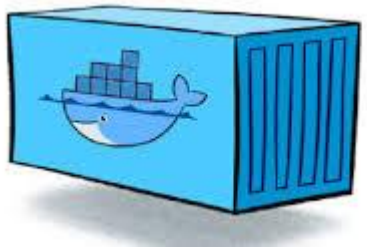
Role of Images and Container

- Images are read-only template composed of layered filesystem used to share common files and create Docker Container instances
- Container –An isolated and secured shipping container created from an image that can be run, started, stopped, moved and deleted



Docker Image

Example: Ubuntu with Node.js and Application Code



An Image is the application we want to run. A Container is an instance of that image running as a process in the memory

Images

- An image is an executable package that includes everything needed to run an application--the code, a runtime, libraries, environment variables and configuration files
- Some of the commands related to images

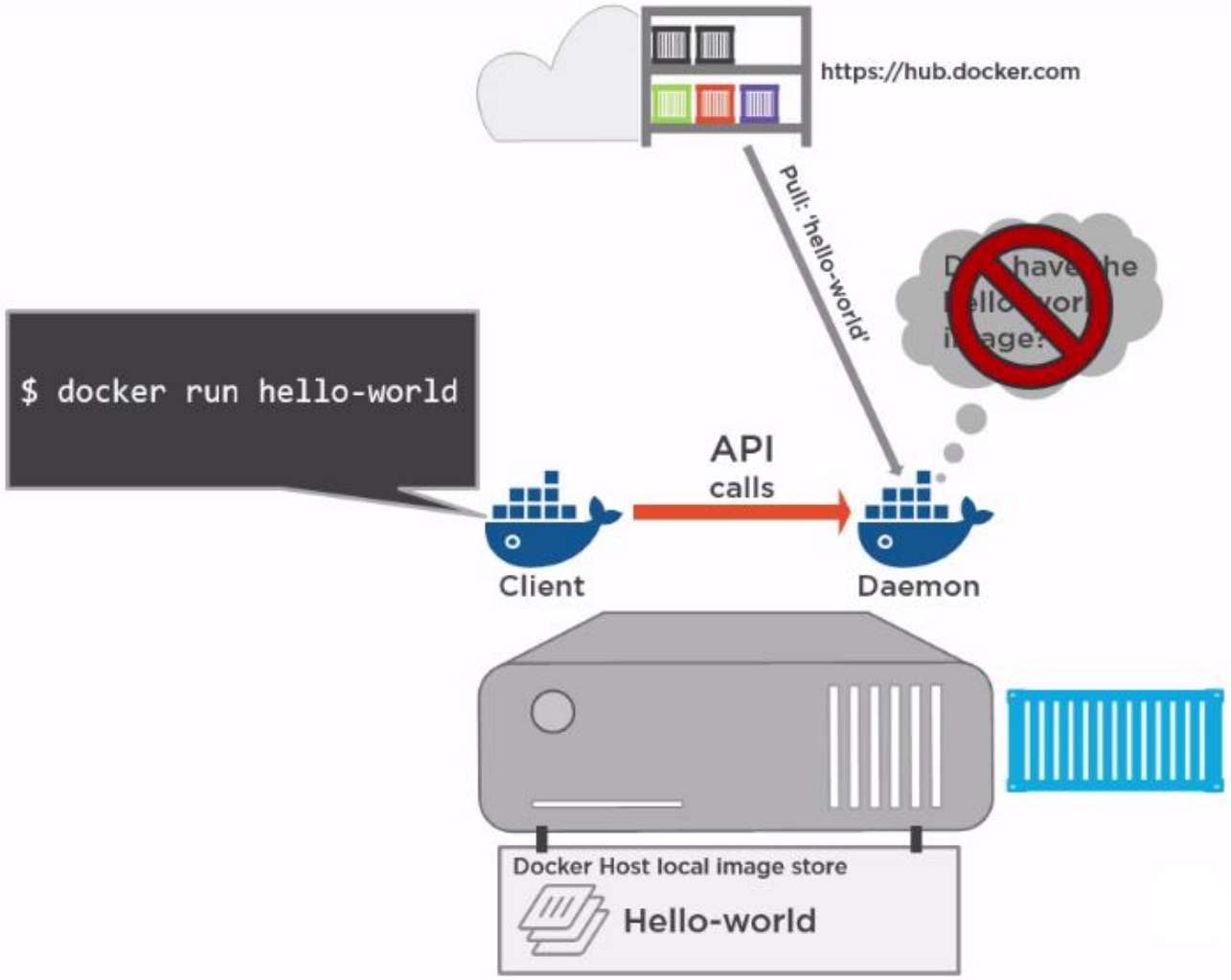
Command	Description
docker images	▪ It will show all top-level images, their repository and tags, and their size ▪ Example: **docker images java** – list all images in the “java” repository ▪ Example: **docker images java:8** – list all images in the “java” with tag “8”
docker rmi <ImageID>	▪ It will remove an image using its short or long ID, its tag, or its digest
docker inspect Repository	▪ This command is used view the details of an image or container. ▪ Example: **docker inspect Jenkins**
docker history <ImageID>	▪ It shows the history of an image
Docker pull	▪ Pull an image or a repository from a registry

Commands to Work with Images

- Table shows commands to work with images

Command	Description
docker images	List all images
docker image history	Show the history of an image
docker image inspect	Display detailed information on one or more images
docker image prune	Remove unused images
docker image pull	Pull an image or a repository from a registry
docker image rm	Remove one or more images
docker image build	Build an image from a Dockerfile

Running Containers



Installing Docker gives you the **client** and **daemon**

Client makes API calls to daemon

Daemon implements the *Docker Remote API*

`docker run` starts a new container

Docker Hub is the default public registry

The daemon will *pull* images that it doesn't already have

https://nellysattari.ghost.io/sitecore-on-docker/

Basic Commands for Containers

Table shows commands to work with containers

Command	Description
docker attach <container-id>	Attach local standard input, output, and error streams to a running container
docker container create	Create a new container
docker container inspect	Display detailed information on one or more containers
docker container logs	Fetch the logs of a container
docker container rm	Remove one or more containers
docker container start	Start one or more stopped containers
docker container stop	Stop one or more running containers
docker container prune	Remove all stopped containers
docker container run –detach <image-name>	Run container in detached mode

How to Create a Custom Image?

- Dockerfile is the building block of Docker images and containers
- Its a list of commands the Docker engine runs to assemble the image, and thus instances of images as containers
- It contains text instructions for building images
- Read by docker image build command
- Read from top, one instruction at a time
- Contains instructions in UPPERCASE

Agenda

- Hypervisors
- Containers and its Benefits
- Docker
- Role of Images and Container
- **Docker Instruction, Hub and Repositories**
- Docker Networking
- Binding and Linking Containers
- Docker Compose and Managing Data in Docker
- Docker Swarm and Service

Docker Instruction (1/6)

LABEL

- You can add labels to your image to help organize images by project, record licensing information, to aid in automation, or for other reasons
- For each label, add a line beginning with LABEL and with one or more key-value pairs

```
# Set one or more individual labels
LABEL com.example.version="0.0.1-beta"
LABEL vendor1="ACME Incorporated"
LABEL vendor2=ZENITH\ Incorporated
LABEL com.example.release-date="2015-02-12"
```

- An image can have more than one label

Docker Instruction (2/6)

RUN

- The RUN instruction will execute any commands in a new layer on top of the current image and commit the results
- The resulting committed image will be used for the next step in the Dockerfile
- It has two forms:

```
RUN <command> (shell form, the command is run in a shell, which by default
is /bin/sh -c on Linux or cmd /S /C on Windows)
or
RUN ["executable", "param1", "param2"]
```

- In the shell form you can use a \ (backslash) to continue a single RUN instruction onto the next line

```
RUN /bin/bash -c 'source $HOME/.bashrc; \ \
echo $HOME'
```

- Most common usage of RUN is apt-get command that is used to install packages
- Always combine RUN apt-get update with apt-get install in the same RUN statement. For example:

```
RUN apt-get update && apt-get install -y \
package-bar \
package-baz \
package-foo
```

Docker Instruction (3/6)

CMD

- The CMD instruction should be used to run the software contained by your image, along with any arguments
- CMD should almost always be used in the form:

```
CMD ["executable", "param1", "param2"…]
```

EXPOSE

- The EXPOSE instruction indicates the ports on which a container listens for connections
- For example, an image containing the Apache web server would use EXPOSE 80 while an image containing MongoDB would use EXPOSE 27017 and so on

Docker Instruction (4/6)

ENV

- To make new software easier to run, you can use ENV to update the PATH environment variable for the software your container installs
- For example, ENV PATH /usr/local/nginx/bin:$PATH ensures that CMD ["nginx"] just works
- The ENV instruction is also useful for providing required environment variables specific to services you wish to containerize
- ENV can also be used to set commonly used version numbers so that version bumps are easier to maintain
- Following snippet shows the use of ENV command

```
FROM ubuntu:15.04
RUN apt-get update && apt-get install -y iputils-ping apache2
ENV var1=ping var2=8.8.8.8
CMD $var1 $var2
```

ADD or COPY

- ADD and COPY are functionally similar
- COPY only supports the basic copying of local files into the container, while ADD has some features (like local-only tar extraction and remote URL support)
- Because image size matters, using ADD to fetch packages from remote URLs is strongly discouraged; you should instead use curl or wget

Docker Instruction (5/6)

ENTRYPOINT

- An ENTRYPOINT allows you to configure a container that will run as an executable
- It has two forms:

```
ENTRYPOINT ["executable", "param1", "param2"](exec form, preferred)
or
ENTRYPOINT command param1 param2 (shell form)
```

- It cannot be overridden at run-time with normal commands such as docker run … <command>
- Any command at run-time is used as an argument to ENTRYPOINT
- Following snippet shows the use of ENTRYPOINT command

```
FROM debian:stable
RUN apt-get update && apt-get install -y --force-yes apache2
EXPOSE 80 443
VOLUME ["/var/www", "/var/log/apache2", "/etc/apache2"] ENTRYPOINT
["/usr/sbin/apache2ctl", "-D", "FOREGROUND"]
```

Docker Instruction (6/6)

Build Image from Dockerfile

- From the directory where the Dockerfile is, simply run:

```
docker build -t="web2"
```

- This command instructs the Docker engine to create a repository named “web2” and tag it with “latest”

Now, to run container, type the command: docker run -d -p 80:80 web2

Difference Between RUN and CMD

- Table shows the difference between RUN and CMD command

CMD	RUN
Run time	Build time
Run commands in container at launch time	Add layers to images
Equivalent to docker run <args> <command> Docker run <args> /bin/bash	Used to install apps
One per docker file	Can be repeated

- Following snippet shows the dockerfile with run and cmd

```
FROM ubuntu:15.04
RUN apt-get update && apt-get install -y iputils-ping
CMD echo "hello world" or CMD echo $var1(shell Form)
```

Docker Hub and Repositories

- Docker Hub serves as a target for your docker push and docker pull commands
- Repository
  - A repository is simply a collection of different versions for a single Docker image
  - In this way, you can think of it as being similar to a git repository
  - For example, the official Ubuntu repository has a description of what is in the image as well as a list of the available versions
- Why Use Registry?
  - Tightly control where your images are being stored
  - Fully own your images distribution pipeline
  - Integrate image storage and distribution tightly into your in-house development workflow

Docker Hub is a cloud-based registry service which allows you to link to code repositories, build your images and test them, stores manually pushed images

Docker – Official Repositories

- Docker Hub contains several Official Repositories
- These are public, certified repositories from vendors and contributors to Docker
- They contain Docker images from vendors like Canonical, Oracle and Red Hat that you can use as the basis to build your applications and services
- Public and Private Repositories
  - You can create public repositories which can be accessed by any other Hub user
  - You can create private repositories with limited access to control
- Docker commands and Docker Hub
  - Docker itself provides access to Docker Hub services via the docker search, pull, login, and push commands

Push an Image to Docker Hub

- You could push to the central docker registry
- The central registry is public
- You can now start pushing to your local repository
- As an example, we will pull the latest postgres image and push version 9.4 to my local registry

```
$ docker pull postgres
$ docker tag postgres:9.4 localhost:5000/postgres:9.4
$ docker push localhost:5000/postgres
```

```
$ docker images
postgres                  9.4                 6dff1b5c2259        5 days ago          244.4 MB
localhost:5000/postgres   9.4                 6dff1b5c2259        5 days ago          244.4 MB
```

Agenda

- Hypervisors
- Containers and its Benefits
- Docker
- Role of Images and Container
- Docker Instruction, Hub and Repositories
- **Docker Networking**
- Binding and Linking Containers
- Docker Compose and Managing Data in Docker
- Docker Swarm and Service

Docker Networking

- In general, a network is a collection of computers or other devices, commonly called nodes, that can communicate with each other
- Docker takes care of the networking aspects so that the containers can communicate with other containers and with the Docker Host
- Docker's networking subsystem is pluggable, using drivers
- Several drivers exist by default, and provide core networking functionality
- By default, Docker provides two network drivers for you, the bridge and the overlay drivers

Default Networks in Docker

- When you install Docker, it creates three networks automatically
- You can list these networks using the docker network ls command

```
[root@ip-172-31-23-19 ec2-user]# docker network ls
NETWORK ID          NAME                DRIVER              SCOPE
1579c5b7ba77        bridge              bridge              local
b665fbaf9d34        host                host                local
f342d03de03e        none                null                local
```

- These three networks are built into Docker
- When you run a container, you can use the --network flag to specify which networks your container should connect to

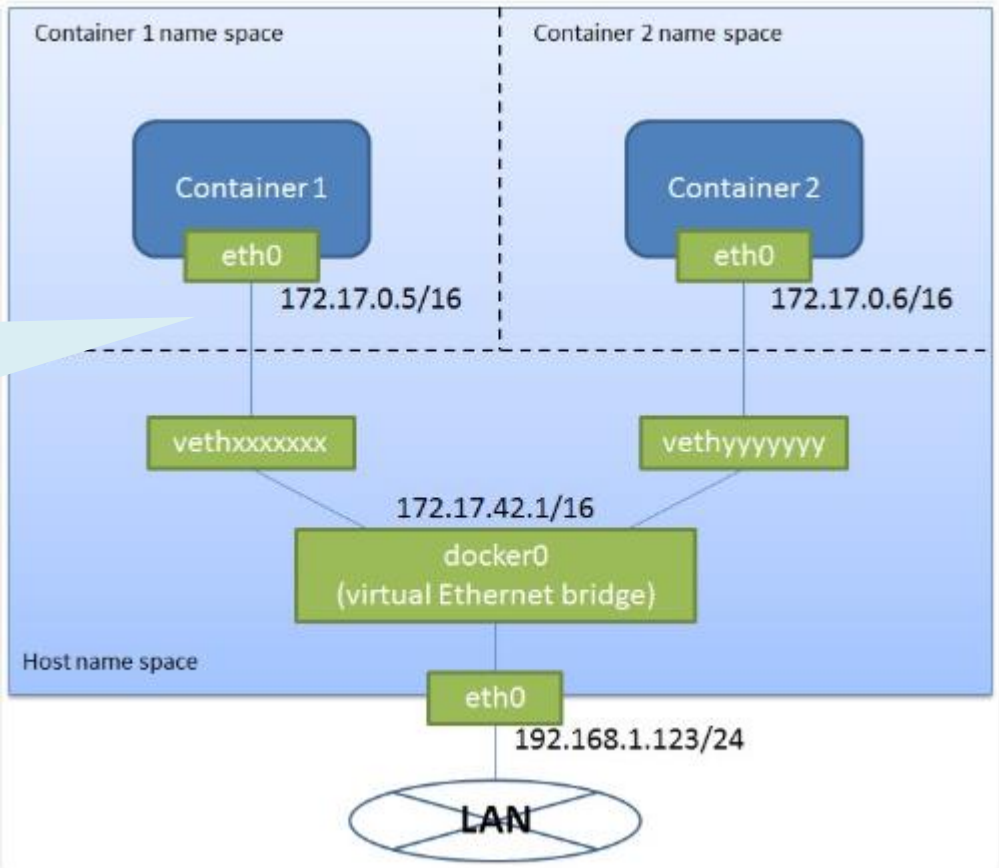
The bridge network represents the docker0

Bridge Network Architecture

- Image shows how Docker containers are connected only to a virtual network Docker

To get details of docker0 bridge:

$ apt-get install bridge-utils
$ brctl show docker0

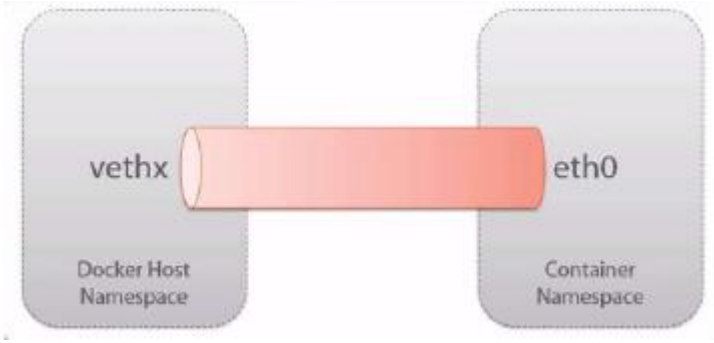


- To inspect containers attached to bridge network - docker network inspect bridge
- To remove container from network - docker network disconnect bridge <container-name>

https://www.kaitoy.xyz/2015/07/25/how-to-capture-packets-on-a-local-network-with-pcap4j-container/

Virtual Interfaces and Bridge

- Image shows how the virtual interfaces are connected to the bridge docker0



- All veth* interfaces are attached/linked to this bridge
- Each veth* interface has a peer (virtual interface) in a specific container

Attach Container to Bridge Network

- One can create a network in Docker before launching containers.
- Syntax:

```
CMD ["executable", "param1", "param2"…]
```

- drivername - Name used for the network driver
- name - Name given to the network
- Example: To create a new network

```
docker network create –-driver  bridge new_network
```

- Now, you can attach the new network when launching the container

```
docker run –it –network=new_network ubuntu:latest /bin/bash
```

Other Network Commands

- Disconnect Container from Network
  - You can disconnect container from specific network any time using the following command

```
CMD ["executable", "param1", "param2"…]
```

- Remove a Docker Network
  - Use rm option to remove any existing unused Docker network. You can specify one or more networks with space separated to remove

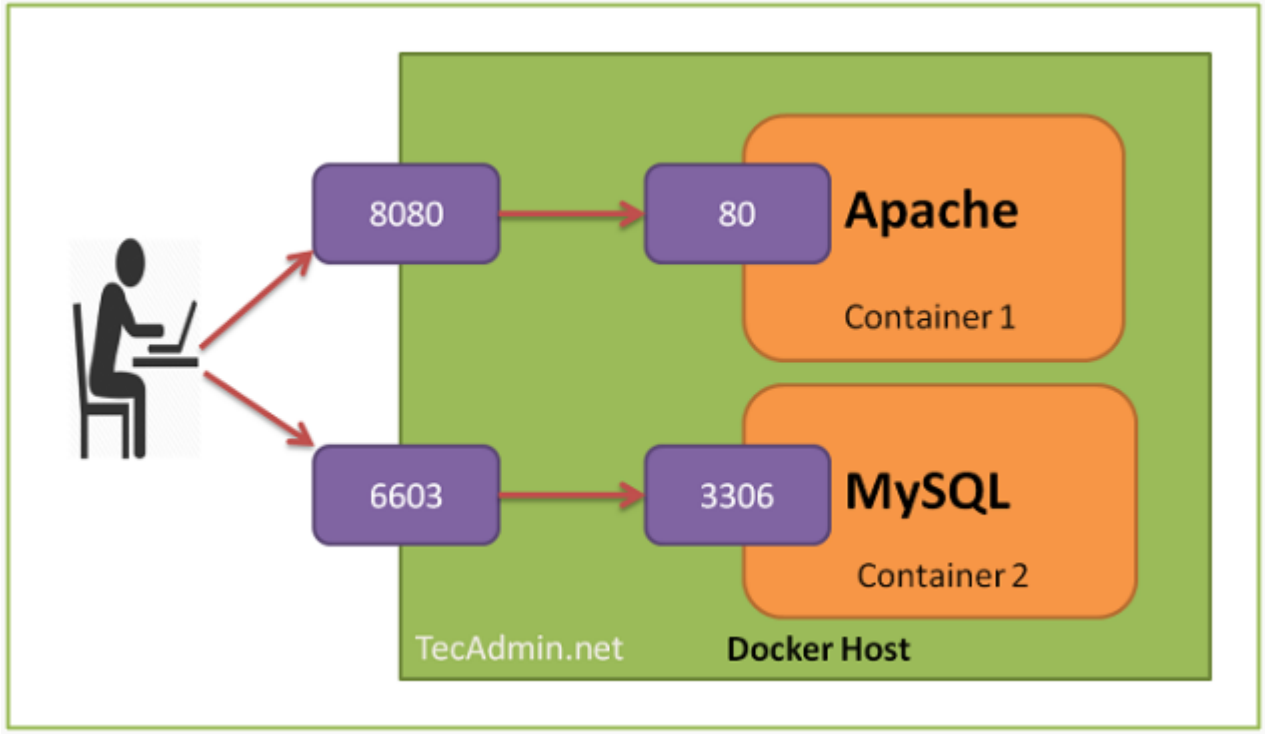
```
docker network rm my-bridge-network network2 network3
```

Agenda

- Hypervisors
- Containers and its Benefits
- Docker
- Role of Images and Container
- Docker Instruction, Hub and Repositories
- Docker Networking
- **Binding and Linking Containers**
- Docker Compose and Managing Data in Docker
- Docker Swarm and Service

Binding Container Ports to Host (1/2)

- By default, Docker containers can make connections to the outside world, but the outside world cannot connect to containers
- Each outgoing connection will appear to originate from one of the host machine's own IP addresses. The Docker server creates a masquerade rule that lets containers connect to IP addresses in the outside world
- Following image shows how to access container applications on ports from the host



https://tecadmin.net/tutorial/docker/docker-manage-ports/

Binding Container Ports to Host (2/2)

- To access the website running on Apache container on port 80
- Steps are:
  - Bind docker host port 8080 to containers port 80
  - Then, you can also use port 80 on docker host as well
  - Command:

```
docker run -it -p 8080:80 apache_image
```

- You can bind multiple ports as well with a single container, but make sure to EXPOSE all ports in Dockerfile before building the image

```
docker network rm my-bridge-network network2 network3
```

Linking Containers (1/4)

- Container Linking allows multiple containers to link with each other
- It is a better option than exposing ports
- Advantage of linking over exposing ports:
  - By linking containers, you provide a secure channel via which Docker containers can communicate to each other
- Example:
  - To link a Web Server container with a Database Server
    - We can launch one Docker container that will be running the Database Server
    - We will launch the second Docker container (Web Server) with a link flag to the container launched in Step 1. This way, it will be able to talk to the Database Server via the link name

Linking Containers (2/4)

- Steps to link Redis container with busybox
  - Pull the redis image
    - **docker pull redis**
  - Launch a redis container in detached mode with the command:
    - **docker run -d --name redis1 redis**
  - Verify the port on which redis server has started with the command: docker ps
    - **docker ps**
  - Now, lets run another container, a busybox:
    - **docker run -it --link redis1:redis --name redisclient1 busybox**
    - The value provided to the **— link** flag is **sourcecontainername:containeraliasname**
    - **redis1** in the **sourcecontainername** since that was the name that was given to our first container that we launched earlier
    - The **containeraliasname** has been selected as **redis** and it could be any name of your choice
    - The above launch of container **(redisclient1)** will lead you to the shell prompt

Linking Containers (3/4)

- Let us observe first what entry has got added in the /etc/hosts file of the redisclient1 container:

```
/ # cat /etc/hosts
127.0.0.1       localhost
::1     localhost ip6-localhost ip6-loopback
fe00::0 ip6-localnet
ff00::0 ip6-mcastprefix
ff02::1 ip6-allnodes
ff02::2 ip6-allrouters
172.17.0.3      redis b30d5be01513 redis1
172.17.0.4      1c35380b1ac0
/ #
```

- Notice an entry at the end, where the container redis1 has got associated with the redis name
- We can ping by the host name as shown:

```
/ # ping redis
PING redis (172.17.0.3): 56 data bytes
64 bytes from 172.17.0.3: seq=0 ttl=255 time=0.113 ms
64 bytes from 172.17.0.3: seq=1 ttl=255 time=0.121 ms
64 bytes from 172.17.0.3: seq=2 ttl=255 time=0.128 ms
64 bytes from 172.17.0.3: seq=3 ttl=255 time=0.106 ms
64 bytes from 172.17.0.3: seq=4 ttl=255 time=0.115 ms
64 bytes from 172.17.0.3: seq=5 ttl=255 time=0.095 ms
64 bytes from 172.17.0.3: seq=6 ttl=255 time=0.128 ms
```

Linking Containers (4/4)

- To print environment variables, use set

```
/ # set
HISTFILE='/root/.ash_history'
HOME='/root'
HOSTNAME='1c35380b1ac0'
IFS=' 
'
LINENO=''
OPTIND='1'
PATH='/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin'
PPID='0'
PS1='\w \$ '
PS2='> '
PS4='+ '
PWD='/'
REDIS_ENV_GOSU_VERSION='1.10'
REDIS_ENV_REDIS_DOWNLOAD_SHA='fc53e73ae7586bcdacb4b63875d1ff04f68c5474c1ddeda78f00e5ae2eed1bbb'
REDIS_ENV_REDIS_DOWNLOAD_URL='http://download.redis.io/releases/redis-4.0.11.tar.gz'
REDIS_ENV_REDIS_VERSION='4.0.11'
REDIS_NAME='/redisclient1/redis'
REDIS_PORT='tcp://172.17.0.3:6379'
REDIS_PORT_6379_TCP='tcp://172.17.0.3:6379'
REDIS_PORT_6379_TCP_ADDR='172.17.0.3'
REDIS_PORT_6379_TCP_PORT='6379'
REDIS_PORT_6379_TCP_PROTO='tcp'
SHLVL='1'
TERM='xterm'
_='clear'
/ #
```

- You can see that various environment variables were auto-created for you to help reach out to the redis1 server from the redisclient1

Agenda

- Hypervisors
- Containers and its Benefits
- Docker
- Role of Images and Container
- Docker Instruction, Hub and Repositories
- Docker Networking
- Binding and Linking Containers
- **Docker Compose and Managing Data in Docker**
- Docker Swarm and Service

Docker Compose (1/2)

- Docker Compose is another best tool for docker to setup multi-container environments
- Using the command, create a single compose file with defining all the containers with their environments
- You can easily use single command to build images and run all the containers
- There is the three-step process to work with Docker Compose
  - Define application environment with Dockerfile for all services
  - Create a docker-compose.yml file defining with all services under application
  - Run **docker-compose up** to run all services under applications

Docker Compose (2/2)

- A docker-compose.yml file required to create to start working with the docker compose
- Sample configuration file of version 3 is as shown:

```
version: '3'
services:
  db:
    image: mysql
    container_name: mysql_db
    restart: always
    environment:
      - MYSQL_ROOT_PASSWORD="secret"
  web:
   image: apache
   build: .
   container_name: apache_web
   restart: always
   ports:
    - "8080:80"
```

Docker Compose SubCommands

- The docker-compose command provides a number of subcommands to manage Docker containers with docker-compose
- Subcommands are:
  - build – The build option is used to build images for services for which build is defined

```
$ docker-compose build          ## Build all services
$ docker-compose build web   ## Build single service
```

  - up – Use to create docker containers with available services in docker-compose.yml file in current directory

```
$ docker-compose up -d                  ## Create all containers
$ docker-compose up -d web             ## Create single container
```

  - Down - This will stop and delete all containers, network and associated images for the services defined in a config file

```
$ docker-compose down            ## Restart all containers
$ docker-compose down web      ## Restart single container
```

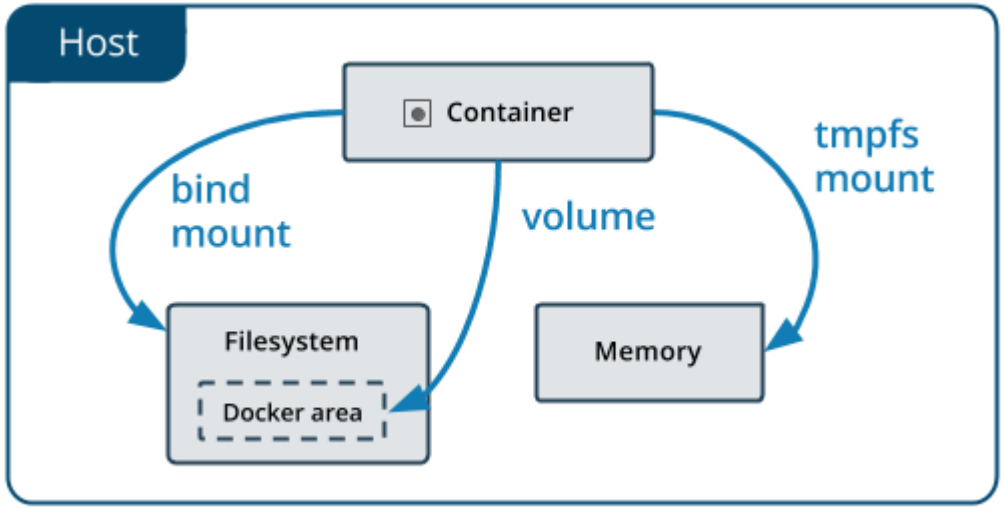
Managing Data in Docker

- By default, all files created inside a container are stored on a writable container layer
- This means that:
  - The data doesn’t persist when that container no longer exists
  - A container’s writable layer is tightly coupled to the host machine where the container is running
  - You can’t easily move the data somewhere else
- Use of Storage Driver
  - Writing into a container’s writable layer requires a storage driver to manage the filesystem
  - The storage driver provides a union filesystem, using the Linux kernel. This extra abstraction reduces performance as compared to using data volumes, which write directly to the host filesystem

Volumes are used to store the data generated by a container so even when it's gone the data it produces still lives on

Volume Types

- There are different type of volumes you can choose from. Mount is about where the data lives on the Docker host



- There are two main types of volumes, **data volumes and bind mounts**

Data Volumes

- Volumes are the preferred mechanism for persisting data generated by and used by Docker containers
- Volumes are completely managed by Docker
- Volumes does not increase the size of the containers using it, and the volume's contents exist outside the lifecycle of a given container
- To mount a volume, use the following command:

```
docker run -d --volumes-from <Container_ID> <YourImage>
```

  - where container ID is id of container you want to mount data from
- Or use --mount switch

```
docker run --mount source=volume-name,destination=/path/in/container,readonly my/image
```

Defining a Data Volume in a Dockerfile

- Data volumes can be defined in a Dockerfile using the VOLUME command
- Following snippet shows how to add volume to Dockerfile named postgres:9.6:

```
ENV PATH $PATH:/usr/lib/postgresql/$PG_MAJOR/bin
ENV PGDATA /var/lib/postgresql/data
RUN mkdir -p "$PGDATA" && chown -R postgres:postgres "$PGDATA" && chmod
777 "$PGDATA" # this 777 will be replaced by 700 at runtime (allows semi-
arbitrary "--user" values)
VOLUME /var/lib/postgresql/data
COPY docker-entrypoint.sh /usr/local/bin/
RUN ln -s usr/local/bin/docker-entrypoint.sh / # backwards compat
ENTRYPOINT ["docker-entrypoint.sh"]
EXPOSE 5432
CMD ["postgres"]
```

- To create a container based on the image:

```
docker run --name my-postgres -e POSTGRES_PASSWORD=password -d
postgres:9.6
```

Changes in the Landscape

- How to scale containers?
- How can containers talk to each other?
- How to ensure high availability of containers?
- How to manage deployments with zero downtime?
- How to control where containers get started?
- How to trust containers?
- How to manage secrets like usernames and passwords?

Few Questions on containers deployment?

Solution – Container Orchestration
The cluster management and orchestration features embedded in the Docker Engine are built using swarmkit

Agenda

- Hypervisors
- Containers and its Benefits
- Docker
- Role of Images and Container
- Docker Instruction, Hub and Repositories
- Docker Networking
- Binding and Linking Containers
- Docker Compose and Managing Data in Docker
- **Docker Swarm and Service**

Orchestration with Docker Swarm

- Docker Swarm is a technique to create and maintain a cluster of Docker Engines or machines
- This enables multiple machines running Docker Engine to participate in a cluster, called Swarm
- The Docker engines contributing to a Swarm are said to be running in Swarm mode
- Machines enter the Swarm mode by either initializing a new swarm or by joining an existing swarm
- A Docker engine participating in a swarm is called a node
- A node can either be a manager node or a worker node
  - Manager Node: The manager node performs cluster management and orchestration
  - Worker Node: The worker node perform tasks allocated by the manager

Manager Node and Worker Node in Swarm (1/2)

Manager Node:

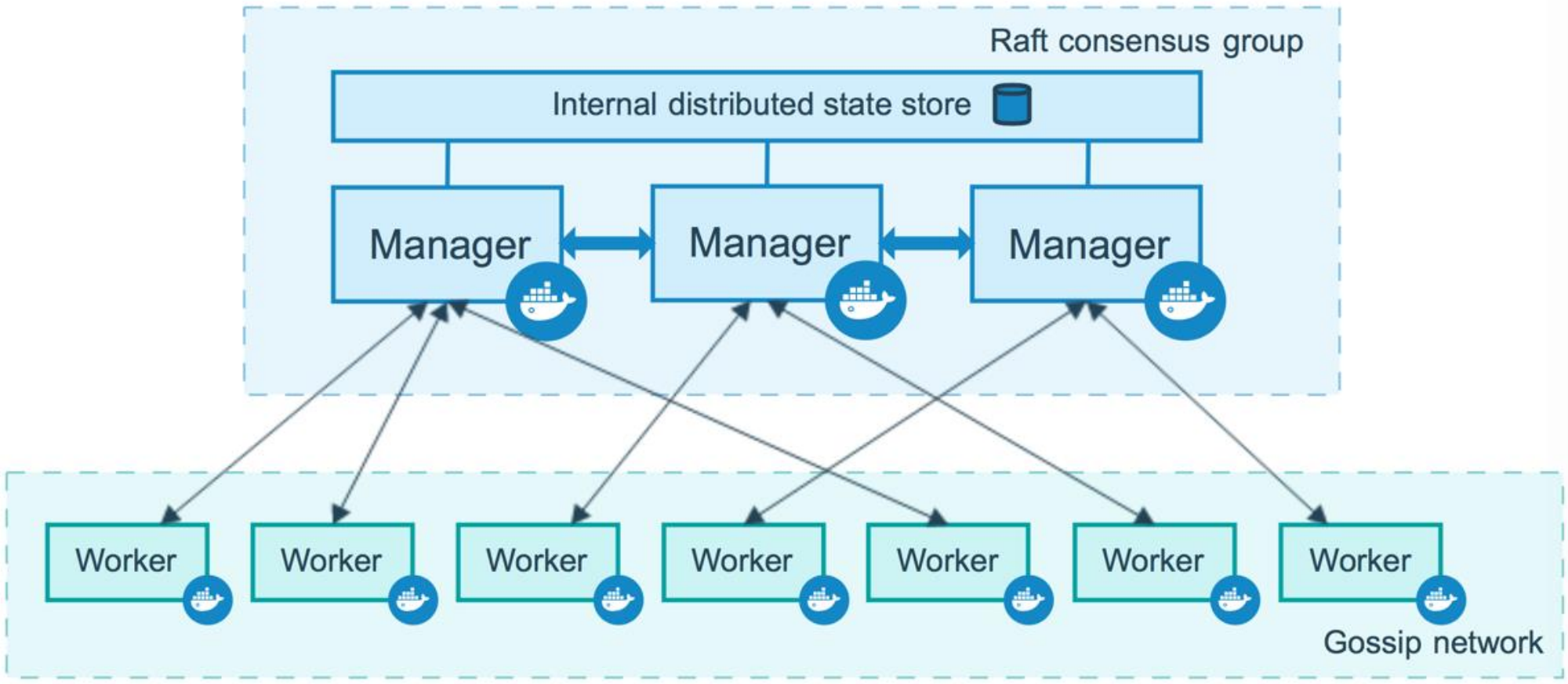
- Docker commands are executed on a cluster by a swarm manager
- Swarm managers can use several strategies to run containers, such as:
  - “emptiest node” -- which fills the least utilized machines with containers
  - “global mode” - which ensures that each machine gets exactly one instance of the specified container
- Swarm managers are the only machines in a swarm that can execute your commands, or authorize other machines to join the swarm as workers
- Enabling swarm mode instantly makes the current machine a swarm manager
- swarm manager natively supports:
  - internal load balancing to distribute tasks across the participating worker nodes
  - ingress load balancing to control exposure of Docker services to the external world
  - service discovery by automatically assigning a DNS entry to every service

Worker Node:

- Workers are just there to provide capacity and aren’t authorize to instruct any machine what it can and cannot do
- There is also an agent on every Worker node, which reports on the state of its tasks to the Manager

Manager Node and Worker Node in Swarm (2/2)

- Following image shows the Manager can maintain the desired state of the cluster



https://docs.docker.com/engine/swarm/how-swarm-mode-works/nodes/

Services and Tasks

The central entity in the Docker Swarm infrastructure is called a service

- A Docker swarm executes services
- The user submits a service to the manager node to deploy and execute
- A service is made up of many tasks
- A task is the most basic work unit in a Swarm
- A task is allocated to each worker node by the manager node
- Services can be scaled at runtime to handle extra load

Docker Swarm Commands

- Docker swarm mode is not enabled by default
- Once docker swarm mode is enabled, then the following commands are available:
  - docker swarm
  - docker node
  - docker service
  - docker stack
  - docker secret

Setting Up Swarm (1/2)

- Docker Swarm is initiated by the manager, or the instance which starts the Swarm cluster becomes the manager
- The command to start the cluster is:

```
docker swarm init --advertise-addr ip-address
```

- Here, the **'--advertise-addr'** flag is used for advertising itself to other nodes who want to join the cluster
- The IP address of the manager needs to be specified along with the flag
- When the Swarm cluster is initiated, a token is generated at the manager's end. This token needs to be used by other nodes to join the swarm cluster
- Example:

```
2. fsoppelsa@yoda: ~ (zsh)
➜ ~ docker swarm init --advertise-addr 192.168.99.100
Swarm initialized: current node (c7m85xhn6ue4rx6zbb86t8rnx) is now a manager.

To add a worker to this swarm, run the following command:
    docker swarm join \
    --token SWMTKN-1-42mf2wtk9m5pctdqiy5bcfqorj9zlnh5n1z3sszoia7l15jwtd-9o4cok8
ewdszlhh8wc826ivvw \
    192.168.99.100:2377

To add a manager to this swarm, run the following command:
    docker swarm join \
    --token SWMTKN-1-42mf2wtk9m5pctdqiy5bcfqorj9zlnh5n1z3sszoia7l15jwtd-bo1vg5g
ilk7lvjwibtx4x0ik0 \
    192.168.99.100:2377
➜ ~
```

Setting Up Swarm (2/2)

- Any node that joins the cluster, can be later promoted to a manager. Execute the below command at the manager's end:

```
docker swarm join \

Or

docker swarm join-token manager
```

- After this, you can run a docker node list command to check how many nodes have joined the cluster along with their status

```
docker node ls
```

Creating a Service

- To start a service, you would need to have the following:
  - Docker image that you want to run. In our case, we will run the standard nginx image that is officially available from the Docker hub
  - We will expose our service on port 80
  - We can specify the number of containers (or instances) to launch. This is specified via the replica's parameter
  - We will decide on the name for our service
- Following snippet shows how to launch 5 instances of the nginx container

```
docker service create --replicas 5 -p 80:80 --name web nginx
```

- To find out the status of service and container replication on various nodes, use the following command

```
docker service ps web
```

Scaling the Service

- To scale containers, use the following command

```
docker service scale web=8
```

- Inspecting Nodes
  - You can inspect the nodes anytime via the docker node inspect command

```
docker node inspect self
```

  - This command is used to inspect the manager node
  - To inspect worker node

```
docker node inspect worker1
```

- Removing the service

```
docker service rm web
```

Applying Rolling Updates to a Service

- You have an updated Docker image to roll out to the nodes

```
docker service update --image <imagename>:<version> web
```

- The scheduler applies rolling updates as follows by default:
  - Stop the first task
  - Schedule update for the stopped task
  - Start the container for the updated task
  - If update to a task returns RUNNING, wait for the specified delay period then start the next task
  - If, at any time during the update, a task returns FAILED, pause the update

Thank you!

Perspective from leaders across the healthcare industry

- On-Demand Webinars
- Reports & eBooks
- Workshops
- Success Stories
- Articles & Blogs
- News

Explore the Knowledge Hub >

CitiusTech

Powering the Future of Healthcare